# Figma → 代码：组件复用验证报告

痛点 4「Figma 到实现存在偏差，组件未被充分复用」· 仓库 luckylucky0001/figma-fe · 2026-10-20 · 交付物：CLI `figmafe`（Go）+ Skill `figma-to-code` + 免费 Figma 导出插件 + Demo + 分步教程

**26** 个真实开源项目（H5 / Android 主流技术栈）× 3 份设计稿

**825/858** 项复用推荐正确，**0** 项错误，其余为「可用但不是最优」

**0/11 → 11/12** 真实项目里实现新页面：不用 Skill → 用 Skill 通过复用检查

**+35%** token、+45% 耗时：用 Skill 多出的成本

**结论**

- **对存量项目，先把「有什么」查清楚，才知道新设计稿能复用什么。** `figmafe reuse` 扫一遍现有 H5（Vue / React / uni-app / Taro + Vant / antd-mobile / NutUI / uni-ui / taro-ui）或 Android（XML View / Compose，QMUI / Material / AppCompat / 自有设计系统）项目，得到组件资产、项目惯例、重复实现和「组件库有但本项目用不了」的组件。`figmafe match` 再把新设计稿逐个元素对上去，给出「直接复用 / 需确认 / 需新建」，并附上项目里的真实写法和变体 → props 的配置。
- **准确率**：26 个项目 × 3 份设计稿共 858 项，✓825 / △33 / ×0。其中 21 个是「留出」项目：调规则时没用过，第一次跑出现过错误（见第 2 节），修的都是通用规则，之后作为回归测试。
- **Agent 实测**：在 5 个真实项目里按同一份设计稿实现新页面，不用 Skill 的 11 次全部没通过复用检查（Now in Android 里新建了 11～12 个和已有组件重复的组件；Haiku 在 newbee 里一个现成组件都没用），用 Skill 的 11/12 次通过。Haiku 从 0/4 提到 4/4：模型越弱，固定流程的收益越大。
- **新页面闭环**：demo 仓库里，`figmafe verify` 一条命令输出复用率、违规、像素差异和节点级偏差，退出码可做 CI 门禁；故意埋的 3 处错误全部被抓出，修正后一次通过。
- **只用免费能力**：不用 Figma Code Connect、Dev Mode MCP 和 Variables REST API（都要付费），设计数据走自己写的插件（Plugin API，免费）。

## 1. 方案：Proposal 的每一条都落成了命令

**① 体检存量**
`reuse`：资产、惯例、重复概念、复制粘贴结构、写死颜色、版本 / 主题兼容性

**② 新稿复用清单**
`match`：Figma 元素 → 项目组件，附真实用法、props 配置、可视化叠图

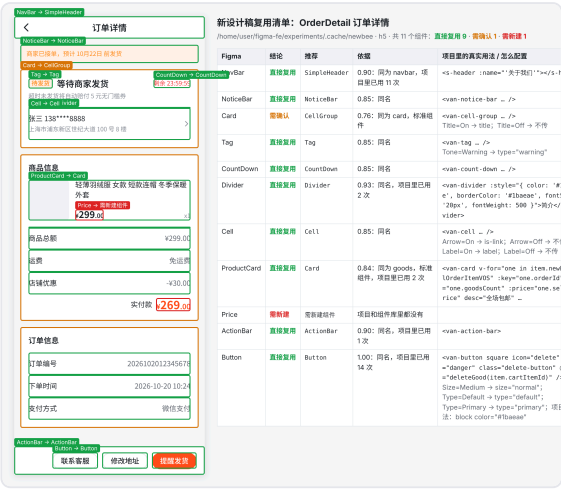
**③ 人确认**
需确认 / 需新建的项写进 `component-map.yaml`，下次自动采用

**④ 实现后检查**
`reuse --focus` / `--changed`：自造轮子、有组件没用、手写已有组件

**⑤ 对照设计**
`compare` / `verify`：截图 + 像素 / 节点级偏差，带 Figma 节点 id

**⑥ 不许变差**
`--baseline --no-regress` 接 CI；偏差沉淀成 `rules.md`

| Proposal 要求 | 落地方式 |
|---|---|
| Figma 组件与代码组件的对应关系 | `match` 按名字、UI 概念（中英文同义词）、项目前缀、使用次数、设计系统封装打分；人确认后写进 `component-map.yaml`，之后 match 优先采用确认过的映射 |
| 开发前识别可复用组件 | 先 `reuse` 再 `match`：只推荐**本项目真能用**的组件（检查组件库版本、改名 / 移除、主题依赖、废弃 API、设计系统是否封装过）；没有组件但有固定写法时推荐「项目惯例」 |
| 开发后检查用了哪些组件、和 Figma 截图对比 | `reuse --focus <页面>` 或 `--changed <git ref>` 只看新增代码；`compare` 截图对比；demo 里 `verify` 把两者合成一个门禁 |
| 偏差反馈给 Agent、沉淀规则 | 每条问题都带文件行号或 Figma 节点 id 和修改建议，Skill 规定照着改到通过；决定和理由写进 component-map，经验写进 `rules.md` |




newbee-mall（Vue 3 + Vant）上跑「订单详情」的复用清单：绿框直接复用、橙框需确认、红框需新建；右侧是项目里的真实写法和 Figma 变体对应的 props。

## 2. 复用清单准不准：26 个真实开源项目（固定 commit，可复现）

| 项目 | 订单详情 | 我的持仓 | 买入下单 |
|---|---|---|---|
| newbee | ✓11 | ✓12 | ✓10 |
| qmui | ✓11 | ✓12 | ✓8 △2 |
| nia | ✓11 | ✓12 | ✓9 △1 |
| antdm-template | ✓11 | ✓12 | ✓10 |
| antdm-mobile | ✓11 | ✓12 | ✓10 |
| jetsnack 留出 | ✓10 △1 | ✓11 △1 | ✓10 |
| litemall 留出 | ✓11 | ✓12 | ✓10 |
| hello-uniapp 留出 | ✓11 | ✓12 | ✓9 △1 |
| taro-ui-demo 留出 | ✓11 | ✓12 | ✓10 |
| nutui-vite 留出 | ✓11 | ✓12 | ✓10 |
| sunflower 留出 | ✓10 △1 | ✓11 △1 | ✓10 |
| pokedex 留出 | ✓11 | ✓12 | ✓10 |
| newpipe 留出 | ✓10 △1 | ✓11 △1 | ✓8 △2 |

| 项目 | 订单详情 | 我的持仓 | 买入下单 |
|---|---|---|---|
| vant-demo-vue3 留出 | ✓11 | ✓12 | ✓10 |
| vant-demo-vue2 留出 | ✓11 | ✓12 | ✓10 |
| vue3-vant-mobile 留出 | ✓11 | ✓12 | ✓10 |
| mall-app-web 留出 | ✓10 △1 | ✓12 | ✓10 |
| antennapod 留出 | ✓10 △1 | ✓11 △1 | ✓9 △1 |
| wanandroid 留出 | ✓11 | ✓12 | ✓10 |
| seal 留出 | ✓10 △1 | ✓11 △1 | ✓9 △1 |
| kunminx 留出 | ✓10 △1 | ✓11 △1 | ✓7 △3 |
| jeecg-uniapp 留出 | ✓11 | ✓12 | ✓10 |
| vue-h5-template 留出 | ✓10 △1 | ✓12 | ✓10 |
| yudao-mall 留出 | ✓10 △1 | ✓10 △2 | ✓10 |
| gkd 留出 | ✓10 △1 | ✓11 △1 | ✓9 △1 |
| androidproject 留出 | ✓10 △1 | ✓11 △1 | ✓10 |

**合计 ✓825 / △33 / ×0**（858 项）。技术栈：Vue 2/3、React、uni-app、Taro × Vant / NutUI / antd-mobile / uni-ui / taro-ui / wot-design-uni / 自有组件；Android XML / Compose × QMUI / Material / AppCompat / 三方 View 库 / 自有设计系统（逐项见 `experiments/real-match-eval.md`）。订单详情是组件化设计稿，我的持仓未组件化、中文图层，买入下单未组件化、含新概念。

✓ 推荐正确；△ 可用但不是最优（多为「需确认」）；× 推荐错误。留出 = 调规则时没用过的项目。

### 留出项目第一次跑的结果（如实记录，修正前）

| 项目 | 第一次 | 出错原因 → 通用修正 |
|---|---|---|
| Jetsnack / litemall / hello-uniapp / NewPipe（第一批） | 3 / 2 / 12 / 3 × | 桌面小部件（Glance）组件被当成 App 组件、没认出 `Jetsnack*` 项目前缀；推荐了 Vant 3 才有、项目装的 Vant 2 没有的 ActionBar；uni-app 内置小写标签被当成项目组件；AppCompat 标准控件没算进资产 → 识别 Glance 和项目前缀、按实际安装版本过滤、加 uni-ui 目录、跨技术栈候选最多「需确认」 |
| mall-app-web / AntennaPod（第二批） | 崩溃 + 1 × / 1 × | uni-app 内置标签没有 import 路径导致崩溃；`<view>` 用量压低了 uni-ui 的占比；AntennaPod 的空状态封装 EmptyViewHandler 不是 View → 修崩溃、占比排除平台内置元素、识别「界面封装类」 |
| WanAndroid / Seal / KunMinX / jeecg（第三批） | 11 / 7 / 3 / 11 × | 三方 View 库和 wot-design-uni 没被收录；Android 的 ActionBar 是顶栏；「Video*」被当成项目前缀；PasswordInputView / ConfirmButton 这类专门用途组件被当成通用组件；会崩溃的 Material 组件没有别的候选时仍排第一 → 逐条改成通用规则 |
| vue-h5-template / 芋道商城 / gkd / AndroidProject（第四批） | 23 × / 崩溃 / 5 × / 8 × | 装了三套 H5 组件库，推荐混用了 Vant 和 NutUI；props 里的三元默认值让扫描崩溃；工具链 `package.json` 让 Android 仓库被当成 H5、图标按钮被当成文字按钮；54 个 AppCompatButton 漏数；「验证码倒计时」被当成倒计时组件 → 定主用组件库、读组件文档注释里的专门用途等通用规则 |
| 买入下单（新设计稿） | 14 个项目 12 × | 开关 / 复选框 / 单选原来是一个概念，搜索算在输入框里 → 拆开概念、补词表、检查 Material 3 主题 |

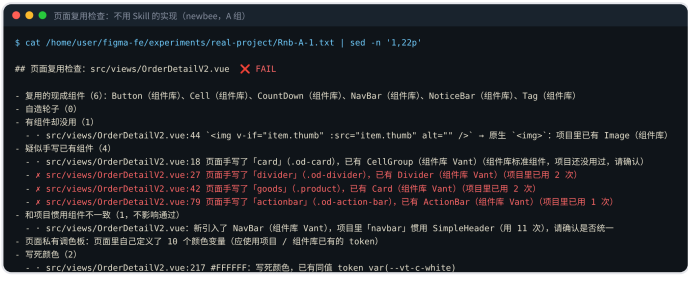
vant-demo、Pokedex、Sunflower、vue3-vant-mobile 第一次跑就没有 ×。复用推荐只错在「项目没见过的结构」，修正都是通用规则，没有针对单个项目的特判；`python3 experiments/real-match-eval.py` 每次改规则都会全量回归。

### 体检：组件库有，不等于本项目能用




QMUI_Android 用的是 AppCompat 主题和 Material 1.4：这些组件会崩溃或编译失败，match 不会推荐。

### 写完检查：有现成组件却手写




不用 Skill 的实现（newbee，A 组）：分割线、商品卡、底部操作栏都是手写的，每条带行号和应复用的组件。

## 3. Agent 实测：真实项目里实现一个新页面

同一份设计稿（订单详情），在 5 个开源项目里新建页面。A 组只给任务和 Figma 导出，C 组加上 Skill（体检 → 复用清单 → 实现 → 页面检查）。所有 run 用同一版 `figmafe reuse --focus / --changed` 打分：自造已有组件、有组件没用、手写已有组件（按 class / id 或横排「标题 + 值」结构识别）任何一项都算不通过。

| 项目 | 技术栈 | A 不用 Skill（Opus / Haiku） | | | C 用 Skill（Opus / Haiku） | | |
|---|---|---|---|---|---|---|---|
| | | 通过 | 复用组件数 | 问题数 | 通过 | 复用组件数 | 问题数 |
| newbee-mall | Vue 3 + Vant | 0/3 | 6 / 6 / 0 | 4 / 4 / 14 | 3/3 | 12 / 12 / 10 | 0 / 0 / 0 |
| litemall（留出） | Vue 2 + Vant 2 | 0/1 | 1 | 14 | 2/2 | 10 / 9 | 0 / 0 |
| QMUI_Android | XML + QMUI | 0/3 | 8 / 9 / 6 | 1 / 1 / 1 | 3/3 | 9 / 8 / 8 | 0 / 0 / 0 |
| NewPipe（留出） | XML + AppCompat | 0/1 | 3 | 2 | 1/1 | 6 | 0 |
| Now in Android | Compose + 自有设计系统 | 0/3 | 2 / 2 / 1 | 6 / 6 / 2 | 2/3 | 7 / 8 / 8 | 2 / 0 / 0 |
| **合计** | | 0/11 | **平均 4.0** | **共 55** | 11/12 | **平均 8.9** | **共 2** |

- **Now in Android**：A 组没先查已有组件，在 core/designsystem 下新建了 NiaNavBar / NiaCard / NiaDivider / NiaCell 等，和已有的 NiaTopAppBar / Card / HorizontalDivider / ListItem 重复，还写了一套 18 个写死颜色的主题。C 组复用了它们，新建的组件都在 component-map.yaml 里写了理由。
- **QMUI / NewPipe**：C 组发现 match 推荐了项目用不了的组件（MaterialDivider 需要 Material 1.5、项目是 1.4；Chip 在 AppCompat 主题下会崩）。这两类问题已经做进工具：体检会列出「组件库有、但本项目不能直接用的组件」，match 不再推荐。实验中 agent 报告的工具问题都已修复并加了回归测试。
- **成本**：有用量记录的 4 次 A、10 次 C，C 平均多 35% token、45% 时间，多在体检、复用清单、页面检查和写决定这几步。

## 4. 新页面实现闭环（demo 仓库）

- 自建的 H5（React）和 Android（Compose，兼容 compileSdk 32 / minSdk 23；另有 XML View 存量组件）组件库加一份订单详情设计稿，跑了 13 次 A / C 对照。`verify` 能区分「页面写错」和「组件本身不符合设计」（见 `docs/component-issues.md`）；教程第 8 步故意写错 3 处，全部被抓出。
- 这个仓库小、组件好找，A 组复用率也是 100%，**证明不了复用率提升**；差别在规范：A 组 7/8 次自造了设计稿外的 CountDown，C 组 0/5。复用率的差距要在真实项目里看（第 3 节）。

## 5. 局限（如实说明）

- **没有真实 Figma 文件**，本环境也访问不了 Figma：三份设计稿由手写的设计源生成（格式和插件导出一致）。插件只用 mock 的 Plugin API 测过，没在真实 Figma 里跑过。
- **没有编译真实项目**：Agent 实验只做静态检查，没有构建和运行这些开源项目（本环境装不了 Android SDK）；demo 的 Android 截图用 Compose Desktop 渲染，不是真机。
- **实验有泄漏**：A 组 agent 也能看到本仓库的 Skill；1 次 A run 自己跑了 figmafe，已剔除。泄漏只会让 A 组更好，所以 A / C 差距是保守估计。样本量小（每格 1～2 次），只能说明方向。
- **评分标准是人定的**：✓ / △ / × 由我逐条判断并写成回归用例；真实接入时应由组件 owner 复核一次。

## 6. 接入你们的项目

| # | 动作 | 产出 |
|---|---|---|
| 1 | `figmafe reuse <H5 / Android 仓库>`（只读，不改代码） | 资产清单、优先处理项、首份体检基线 |
| 2 | Figma 插件导出下一个要做的页面，`figmafe match` | 复用清单 + 叠图，组件 owner 确认「需确认」项 |
| 3 | 开发时让 Agent 按 Skill 走；提交前 `reuse --changed origin/main` | 新增代码的复用问题，带行号 |
| 4 | CI 里 `reuse --baseline main.json --no-regress`（示例：`docs/ci/`） | 组件化率、写死颜色等指标只许变好 |

教程：`docs/tutorial/README.md` · 数据：`experiments/real-match-eval.md`、`match-judgement.md`、`real-project/summary.md`